

Vel Tech Group of Institutions

Name: GOPICHAND KAKIVAI

Email: vtu29748@veltech.edu.in

Roll no: vtu29748

Phone: null

Branch: VTU

Department: CSE

Batch: 2028

Degree: B.Tech - Computer Science and Engineering

Scan to verify results



2028_NeoColab_Design and Analysis of AlgorithmsS1L3

VTU_DAA_Week 4_PAH

Attempt : 1

Total Mark : 50

Marks Obtained : 48.5

Section 1 : Coding

1. Problem Statement

Janu is tasked with creating a program that solves the Traveling Salesman Problem (TSP). The goal is to find the shortest possible route that visits a set of cities and returns to the starting city.

You are required to implement a program that takes input regarding the number of cities and the distance matrix between cities and outputs the optimal route and its associated minimum cost.

The diagonal values will be 0, as the distance from a city to itself is 0.

Note: Solve it using Branch and Bound approach.

Input Format

The first line of input consists of an integer n , representing the number of cities.

The next n lines contain n space-separated integers each, representing the distance matrix between the cities.

The cell (i, j) represents the distance from city i to city j .

Output Format

The first line of output displays "The Path is: " followed by the path of the route using ---> in between.

The second line displays "Minimum cost is " followed by the total minimum cost associated with the shortest route.

Refer to the sample output for the formatting specifications.

Sample Test Case

Input: 4

0 4 1 3

4 0 2 1

1 2 0 5

3 1 5 0

Output: The Path is: 1 ---> 3 ---> 2 ---> 4 ---> 1

Minimum cost is 7

Answer

```
#include <stdio.h>
```

```
#include <stdlib.h>
```

```
#include <limits.h>
```

```
int n;
```

```
int dist[15][15];
```

```
int bestCost=INT_MAX;
```

```
int bestPath[15];
```

```
void tspUtil(int path[],int visited[],int level,int cost){
```

```
    if(level==n){
```

```
        cost+=dist[path[level-1]][path[0]];
```

```
        if(cost<bestCost){
```

```

        bestCost=cost;
        for(int i=0;i<n;i++) bestPath[i]=path[i];
    }
    return;
}
for(int i=0;i<n;i++){
    if(!visited[i]){
        int newCost=cost+dist[path[level-1]][i];
        if(newCost<bestCost){
            path[level]=i;
            visited[i]=1;
            tspUtil(path,visited,level+1,newCost);
            visited[i]=0;
        }
    }
}
}
}

```

```

int main(){
    scanf("%d",&n);
    for(int i=0;i<n;i++)
        for(int j=0;j<n;j++)
            scanf("%d",&dist[i][j]);

    int path[15],visited[15]={0};
    path[0]=0;
    visited[0]=1;
    tspUtil(path,visited,1,0);

    printf("The Path is: ");
    for(int i=0;i<n;i++){
        printf("%d",bestPath[i]+1);
        printf("--->");
    }
    printf("%d\n",bestPath[0]+1);

    printf("Minimum cost is %d\n",bestCost);
    return 0;
}

```

Status : Partially correct

Marks : 8.5/10

2. Problem Statement

Karan is planning a delivery route that starts from city 0, visits every other city exactly once, and returns back to city 0.

The travel costs between cities are given as a cost matrix.

Your task is to find the minimum possible travel cost and print the optimal route.

Cost Formula

For a route:

0 a b ... 0

Total Cost = $\text{cost}[0][a] + \text{cost}[a][b] + \dots + \text{cost}[\text{last}][0]$

Input Format

The first line contains an integer n , representing the number of cities.

The next n lines contain n space-separated integers representing the cost matrix.

Output Format

The output prints multiple lines showing path exploration in the format:

The first line prints the minimum travel cost.

The second line prints the optimal path starting from city 0 and ending at city 0.

Refer to the sample output for the formatting specifications.

Sample Test Case

Input: 4
0 10 15 20
10 0 35 25
15 35 0 30
20 25 30 0

Output: 80
0 1 3 2 0

Answer

```
#include <stdio.h>
#include <stdlib.h>
#include <limits.h>
```

```
int n;
int cost[10][10];
int bestCost=INT_MAX;
int bestPath[10];
```

```
void tspUtil(int path[],int visited[],int level,int currentCost){
    if(level==n){
        currentCost+=cost[path[level-1]][path[0]];
        if(currentCost<bestCost){
            bestCost=currentCost;
            for(int i=0;i<n;i++) bestPath[i]=path[i];
        }
        return;
    }
    for(int i=1;i<n;i++){
        if(!visited[i]){
            int newCost=currentCost+cost[path[level-1]][i];
            if(newCost<bestCost){
                path[level]=i;
                visited[i]=1;
                tspUtil(path,visited,level+1,newCost);
                visited[i]=0;
            }
        }
    }
}
```

```
int main(){
    scanf("%d",&n);
```

```

for(int i=0;i<n;i++)
    for(int j=0;j<n;j++)
        scanf("%d",&cost[i][j]);

int path[10],visited[10]={0};
path[0]=0;
visited[0]=1;
tspUtil(path,visited,1,0);

printf("%d\n",bestCost);
for(int i=0;i<n;i++) printf("%d ",bestPath[i]);
printf("0\n");
return 0;
}

```

Status : Correct

Marks : 10/10

3. Problem Statement

You are a logistics manager responsible for planning delivery routes for a fleet of vehicles. Each vehicle needs to visit a set of locations to deliver goods, and the cost of traveling between locations is known.

Your goal is to find the shortest route for each vehicle, ensuring that each location is visited exactly once before returning to the starting point.

Note: Solve it using Branch and Bound approach.

Input Format

The first line contains an integer n , the number of locations.

The next n lines contain n integers each, representing the cost matrix of traveling between locations.

The i th integer on the j th line represents the cost of traveling from location i to location j .

Output Format

The first line of output displays "Shortest Path: " followed by the shortest path that visits all cities.

The second line of output displays "Minimum cost is " followed by the minimum cost.

Refer to the sample output for the formatting specifications.

Sample Test Case

Input: 5

0 10 8 9 7

10 0 10 5 6

8 10 0 8 9

9 5 8 0 6

7 6 9 6 0

Output: Shortest Path: 1 3 4 2 5 1

Minimum cost is 34

Answer

```
#include <stdio.h>
```

```
#include <stdlib.h>
```

```
#include <limits.h>
```

```
int n;
```

```
int cost[15][15];
```

```
int bestCost=INT_MAX;
```

```
int bestPath[15];
```

```
void tspUtil(int path[],int visited[],int level,int currentCost){
```

```
    if(level==n){
```

```
        currentCost+=cost[path[level-1]][path[0]];
```

```
        if(currentCost<bestCost){
```

```
            bestCost=currentCost;
```

```
            for(int i=0;i<n;i++) bestPath[i]=path[i];
```

```
        }
```

```
        return;
```

```
    }
```

```
    for(int i=1;i<n;i++){
```

```
        if(!visited[i]){
```

```
            int newCost=currentCost+cost[path[level-1]][i];
```

```
            if(newCost<bestCost){
```

```

        path[level]=i;
        visited[i]=1;
        tspUtil(path,visited,level+1,newCost);
        visited[i]=0;
    }
}
}

int main(){
    scanf("%d",&n);
    for(int i=0;i<n;i++)
        for(int j=0;j<n;j++)
            scanf("%d",&cost[i][j]);

    int path[15],visited[15]={0};
    path[0]=0;
    visited[0]=1;
    tspUtil(path,visited,1,0);

    printf("Shortest Path: ");
    for(int i=0;i<n;i++) printf("%d ",bestPath[i]+1);
    printf("\n");

    printf("Minimum cost is %d\n",bestCost);
    return 0;
}

```

Status : Correct

Marks : 10/10

4. Problem Statement

Michael has been given an important task—developing a program to solve the classic 0/1 Knapsack Problem. His goal is to help users select the most valuable combination of items while staying within a specified weight limit.

In this problem, each item has a value and a weight, and the objective is to maximize the total value without exceeding the given weight capacity.

Michael's program should determine the maximum possible value that can be obtained by selecting the optimal set of items.

Input Format

The first line of input consists of an integer n , representing the number of items available for selection.

The second line of input consists of n space-separated integers, P , which represent the values of the items.

The third line of input consists of n space-separated integers, C , which represent the weights of the corresponding items.

The fourth line of input consists of an integer W , representing the maximum weight limit of the knapsack.

Output Format

The output displays the maximum value that can be obtained using the knapsack algorithm in the following format: "Maximum value: [maximum value]"

Refer to the sample outputs for the exact format.

Sample Test Case

Input: 6
300 150 120 100 90 80
6 3 3 2 2 2
10

Output: Maximum value: 490

Answer

```
#include <stdio.h>
```

```
int max(int a,int b){return a>b?a:b;}
```

```
int main(){  
    int n,W;  
    scanf("%d",&n);  
    int P[n],C[n];
```

```

for(int i=0;i<n;i++) scanf("%d",&P[i]);
for(int i=0;i<n;i++) scanf("%d",&C[i]);
scanf("%d",&W);

int dp[n+1][W+1];
for(int i=0;i<=n;i++){
    for(int w=0;w<=W;w++){
        if(i==0||w==0) dp[i][w]=0;
        else if(C[i-1]<=w)
            dp[i][w]=max(P[i-1]+dp[i-1][w-C[i-1]],dp[i-1][w]);
        else
            dp[i][w]=dp[i-1][w];
    }
}

printf("Maximum value: %d\n",dp[n][W]);
return 0;
}

```

Status : Correct

Marks : 10/10

5. Problem Statement

Ragul is interested in solving the knapsack problem. He has a list of n items, each with a weight and a value. Ragul wants to determine the maximum value he can obtain by selecting a combination of items to fit into his knapsack, which has a maximum weight capacity.

Help Ragul write a program that calculates the following:

Calculate and print the sum of all n values. Calculate and print the average value of all n values with exactly 2 decimal places. Calculate and print the maximum value that can be obtained by selecting a combination of items to fit into the knapsack without exceeding its capacity.

Note: Use the 0/1 knapsack method to solve the problem.

Input Format

The first line of input contains an integer n , representing the number of items.

The next line contains n space-separated integers, representing the weights of the items.

The next line contains n space-separated integers, representing the values of the items.

The last line contains an integer C, representing the maximum weight capacity of the knapsack.

Output Format

The output consists of the following in each line:

1. The sum of all n values as an integer.
2. The average value of all n values with exactly 2 decimal places is a double value.
3. The maximum value that can be obtained as an integer.

Refer to the sample output for the formatting specifications.

Sample Test Case

Input: 3
10 15 20
45 65 70
30

Output: Sum of values: 180
Average of values: 60.00
Maximum amount: 115

Answer

```
#include <stdio.h>
```

```
int max(int a,int b){return a>b?a:b;}
```

```
int main(){  
    int n,C;  
    scanf("%d",&n);  
    int weights[n],values[n];  
    for(int i=0;i<n;i++) scanf("%d",&weights[i]);  
    for(int i=0;i<n;i++) scanf("%d",&values[i]);
```

```

scanf("%d",&C);

int sum=0;
for(int i=0;i<n;i++) sum+=values[i];
double avg=(double)sum/n;

int dp[n+1][C+1];
for(int i=0;i<=n;i++){
    for(int w=0;w<=C;w++){
        if(i==0||w==0) dp[i][w]=0;
        else if(weights[i-1]<=w)
            dp[i][w]=max(values[i-1]+dp[i-1][w-weights[i-1]],dp[i-1][w]);
        else
            dp[i][w]=dp[i-1][w];
    }
}

printf("Sum of values: %d\n",sum);
printf("Average of values: %.2f\n",avg);
printf("Maximum amount: %d\n",dp[n][C]);
return 0;
}

```

Status : Correct

Marks : 10/10